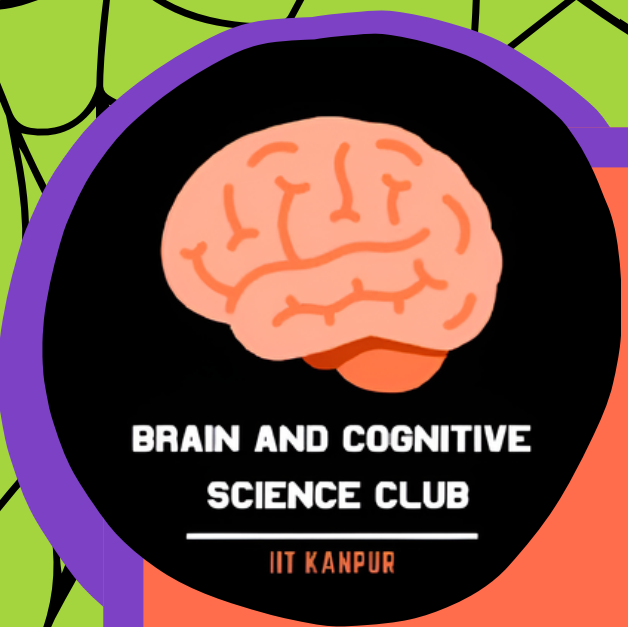


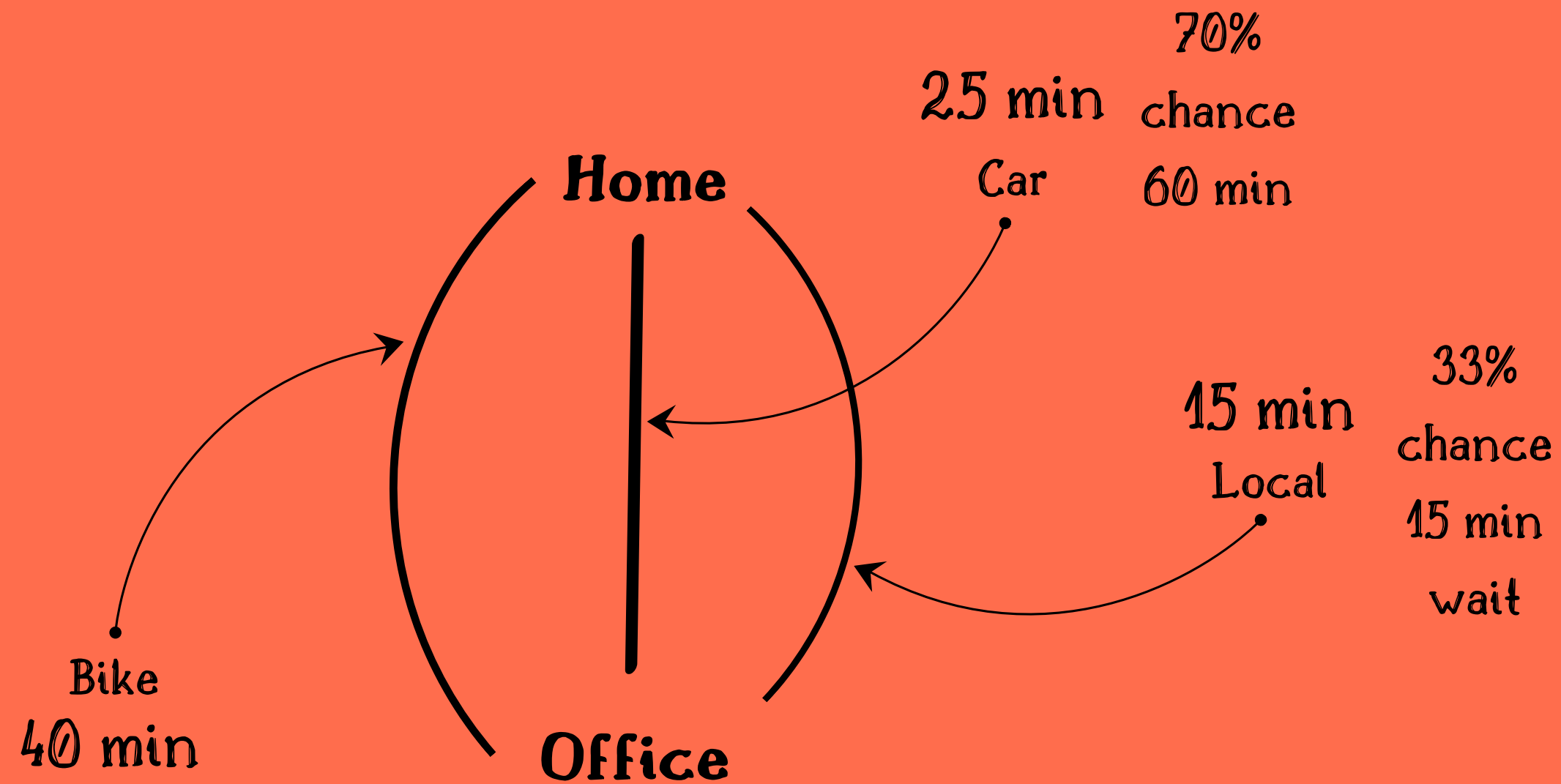


REINFORCEMENT LEARNING

Recall- ϵ



WHY REINFORCEMENT LEARNING?







MARKOV DECISION PROCESS

State Space

The set of all possible situations in which the agent can exist.

Transition Probabilities

The likelihood of moving from one state to another based on actions.

Discount Factor

The present value of future rewards.



Action Space

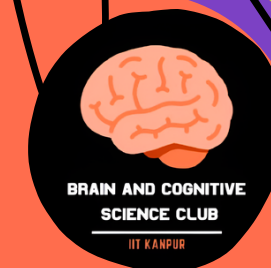
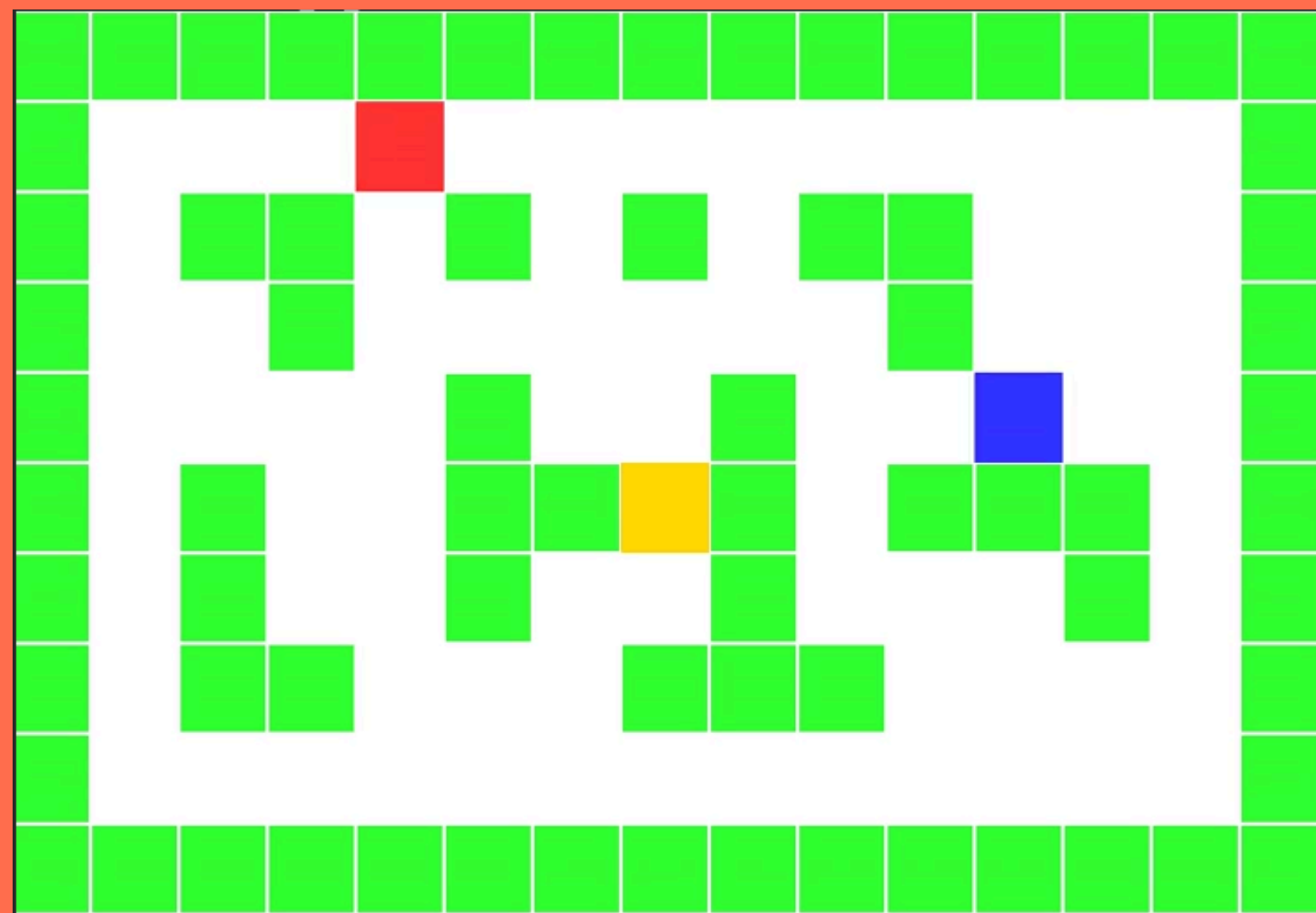
The set of choices available to the agent at each state.

Reward Function

The immediate benefit received after taking an action.

REWARD STRUCTURE

Rewards?



EXPLORATION VS EXPLOITATION

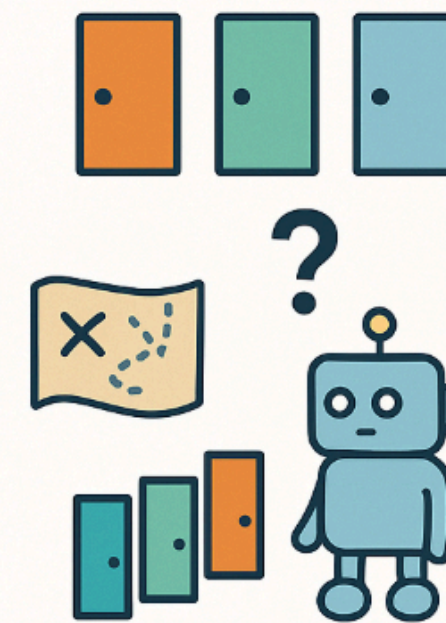


- **Exploration** = trying out new actions (or policies) to gather information about the environment
- **Exploitation** = using the best known action/policy so far to maximise immediate reward, based on what you've learnt.

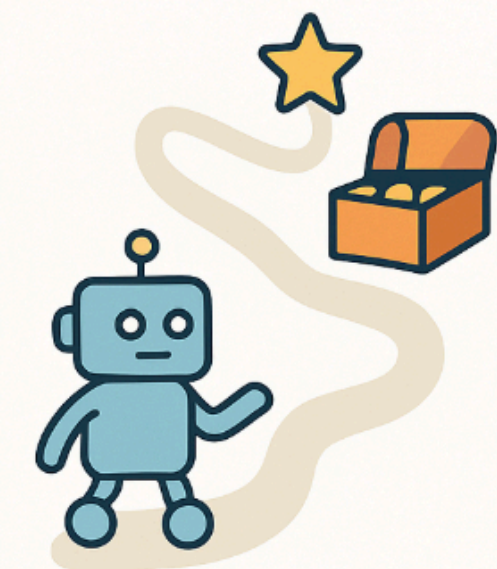
Why the trade-off matters:

- Pure exploration → wasted opportunities (you don't leverage what you already know).
- Pure exploitation → risk of getting stuck in a sub-optimal policy because you never try alternatives.
- The goal: balance so that you learn enough while also earning good reward.

Exploration vs Exploitation in Reinforcement Learning



Explore



Exploit



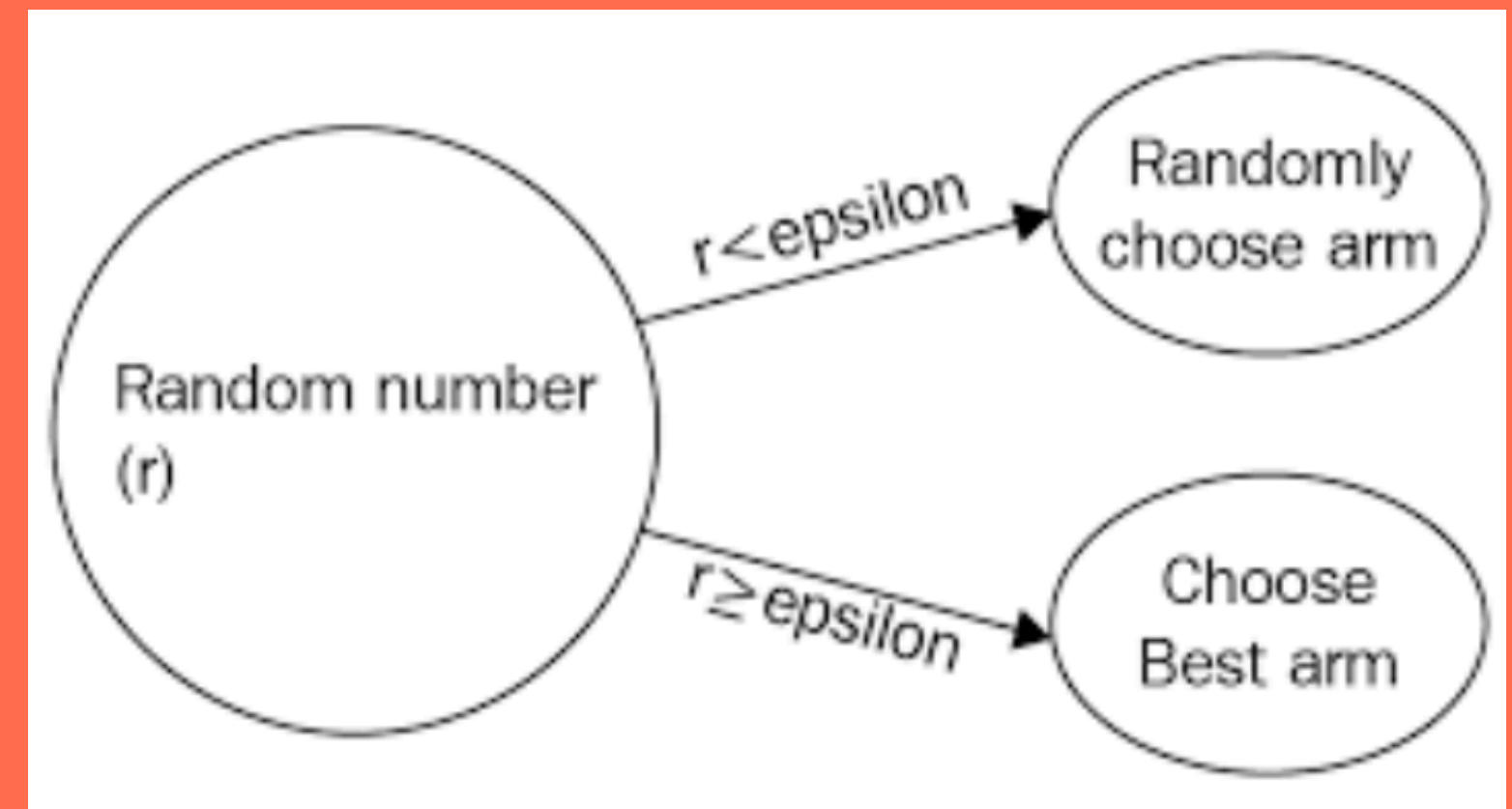
THE ϵ -GREEDY APPROACH

What is ϵ -Greedy?

- With probability ϵ you explore (choose a random action); with probability $1 - \epsilon$ you exploit (choose the best action so far).

Key trade-offs / notes:

- Larger $\epsilon \rightarrow$ more exploration (better long-term knowledge) but lower immediate reward.
- Smaller $\epsilon \rightarrow$ more exploitation (higher immediate reward) but risk of missing better actions.
- Decay schedule is important: often start high ϵ , reduce as learning progresses.



BELLMAN'S EQUATION

Value Function

$$V(s) = \max_a \sum P(s'|s,a)[R(s,a,s') + \gamma V(s')]$$

Action-Value Function

$$Q(s,a) = \sum P(s'|s,a)[R(s,a,s') + \gamma \max_{a'} Q(s',a')]$$

Optimal Policy

$$\pi^*(s) = \operatorname{argmax}_a Q^*(s,a) - \text{choose action with highest } Q\text{-value}$$



BELLMAN'S EQUATION

What we genrally use

$$Q(s,a) = R(s,a) + \gamma \max_{a'} Q(s',a')$$





Q-LEARNING

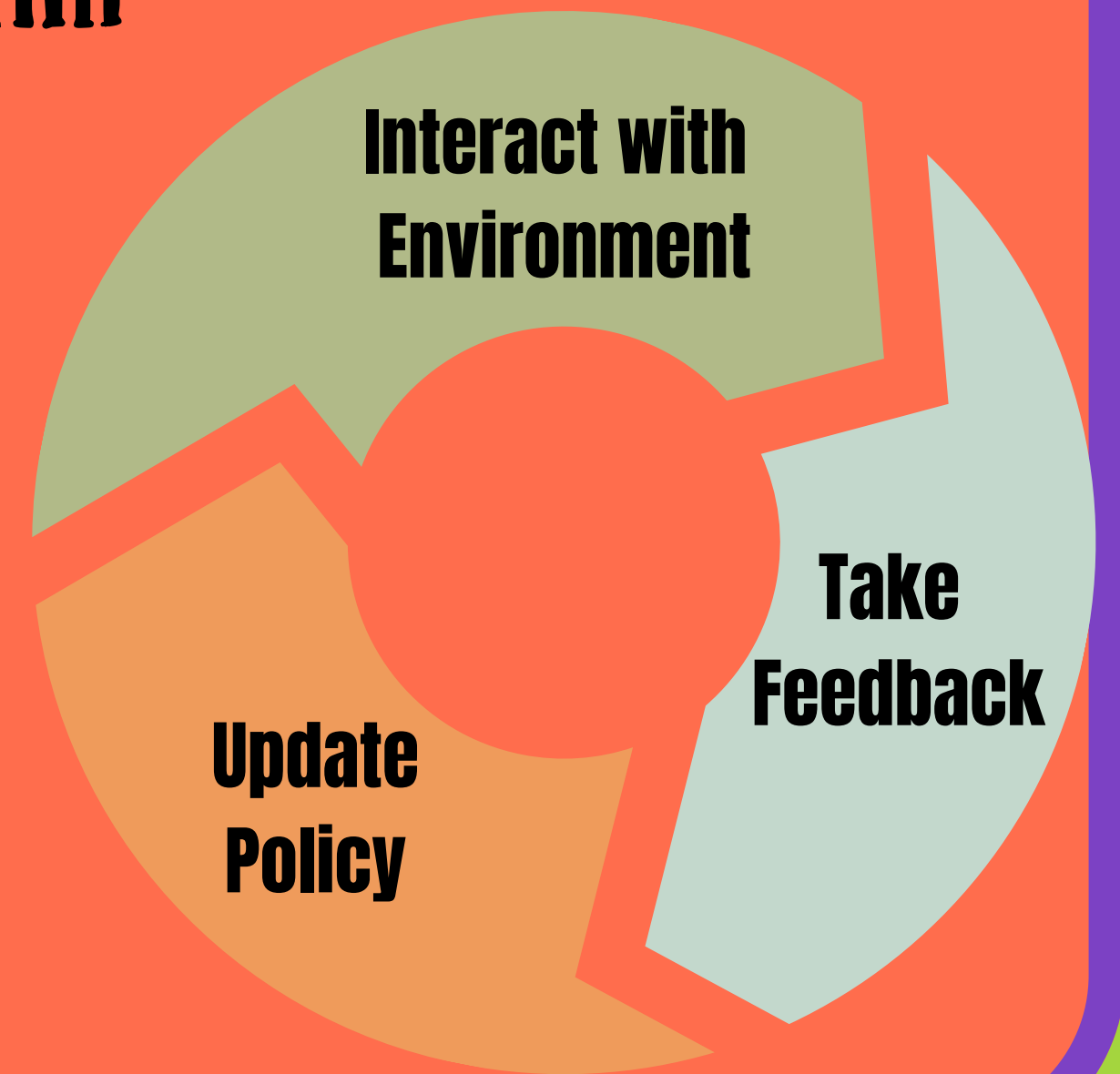
You first RL Algorithm

It's a type of machine learning where a computer learns to make smart decisions by learning from its mistakes, just like we do.

But isn't that what all ML Algorithms do ??

Yes, but It differs from others because:

- To begin with you have an environment to interact with instead of a ground truth dataset.
- In Q learning you aren't training a model, you train an agent to develop a decision making policy.



HOW DOES IT WORK



Your goal is to create a cheate sheet for all possible state and action pair, so essentially you have to make a table called “Q-table”.



Q-LEARNING

Initialize Q-table with arbitrary values (often zeros)

$Q(\text{terminal state}, \cdot) = 0$.

Initialize parameters: learning rate (α), discount factor (γ).

For each episode:

Initialize the starting state (s).

Repeat (for each step of the episode) until s is terminal:

With probability epsilon choose random action(a) (exploration)

or

Choose an action (a) from state (s) with the maximum Q-value (exploitation).

Take action (a), observe the immediate reward (r) and the new state (s').

Update the Q-table entry $Q(s, a)$ using the Q-learning update rule:

$$Q(s, a) \leftarrow (1-\alpha) Q(s, a) + \alpha * [r + \gamma * \max_{a'} Q(s', a')]$$

Set the new state as the current state: $s \leftarrow s'$

